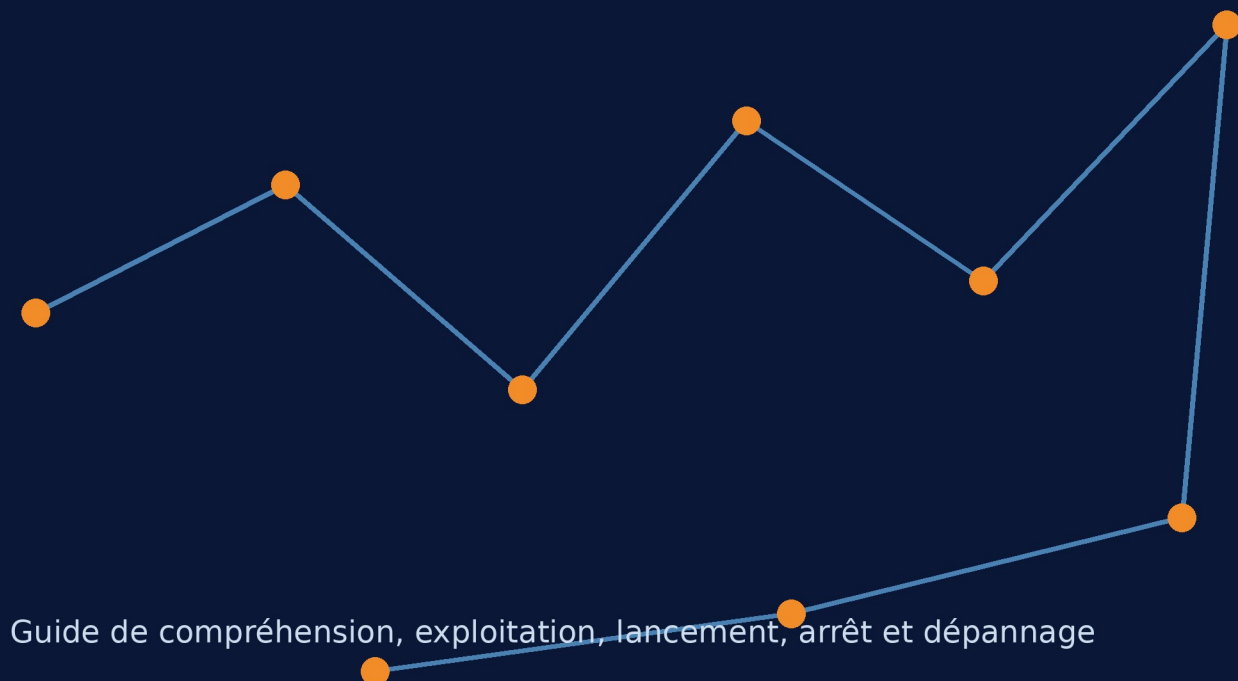
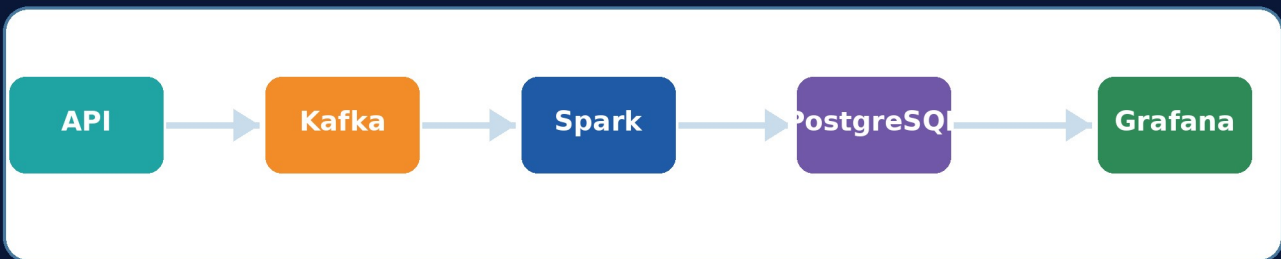


PIPELINE DATA ENGINEERING EN TEMPS REEL

Manuel pratique complet

Open-Meteo - Kafka - Spark Structured Streaming
Airflow - PostgreSQL - Grafana - Docker



À propos de ce manuel

Un cours pratique pour comprendre, exploiter et présenter ton pipeline de données en temps réel.

Ce document est conçu comme un manuel personnel. Il ne cherche pas à recopier tout le code : il explique surtout ce que fait chaque composant, comment démarrer et arrêter le projet, comment vérifier qu'il fonctionne, comment diagnostiquer un problème, et comment raconter le projet en entretien.

Le résultat final

Tu disposes d'une chaîne complète : une API publique est interrogée régulièrement, chaque observation devient un événement Kafka, Spark la transforme, PostgreSQL la stocke, Grafana l'affiche et Airflow supervise le fonctionnement.

Accès rapides du projet

Airflow: <http://localhost:8080> - utilisateur airflow / mot de passe airflow, sauf modification

Kafka UI: <http://localhost:8081>

Spark UI: <http://localhost:4040> pendant que le job Spark est actif

Grafana: <http://localhost:3001> - utilisateur admin / mot de passe admin, sauf modification

PostgreSQL depuis Windows: localhost:5433 - base weather - utilisateur weather_user

PostgreSQL depuis Docker: weather-db:5432

Comment utiliser le document

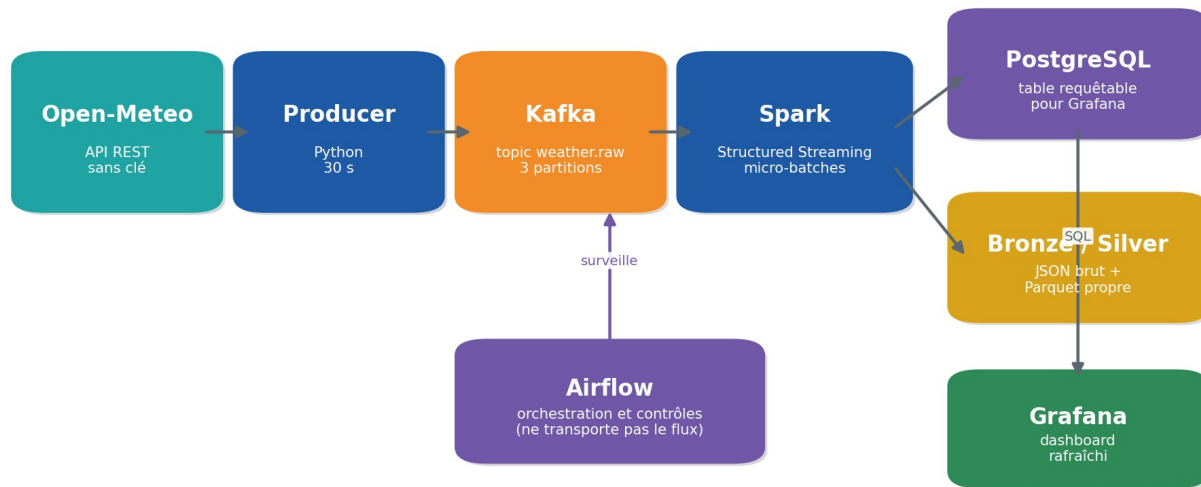
- **Pour relancer rapidement :** va directement au chapitre 7 et à la fiche mémo finale.
- **Pour comprendre :** lis les chapitres 1 à 6 dans l'ordre.
- **Pour dépanner :** commence par le chapitre 13 et la commande docker compose ps.
- **Pour préparer un entretien :** utilise les chapitres 15 et 16.

Sommaire

Chapitre	Contenu	Page
1	Vue d'ensemble du projet	5

Chapitre	Contenu	Page
2	Notions fondamentales du Data Engineering	7
3	Architecture et cycle de vie d'une donnée	9
4	Docker et Docker Compose	11
5	Kafka : transport des événements	14
6	Spark Structured Streaming et architecture Bronze/Silver	16
7	Lancer le projet	18
8	Exploiter, arrêter et redémarrer	20
9	PostgreSQL, DBeaver et SQL utile	22
10	Grafana et dashboards temps réel	24
11	Airflow et orchestration	26
12	Vérifier le pipeline de bout en bout	28
13	Dépannage complet	30
14	Qualité, doublons, sécurité et bonnes pratiques	32
15	Couche Gold et prochaines évolutions	34
16	Présenter le projet sur GitHub et en entretien	36
17	Fiches mémo et glossaire	38

Architecture du projet



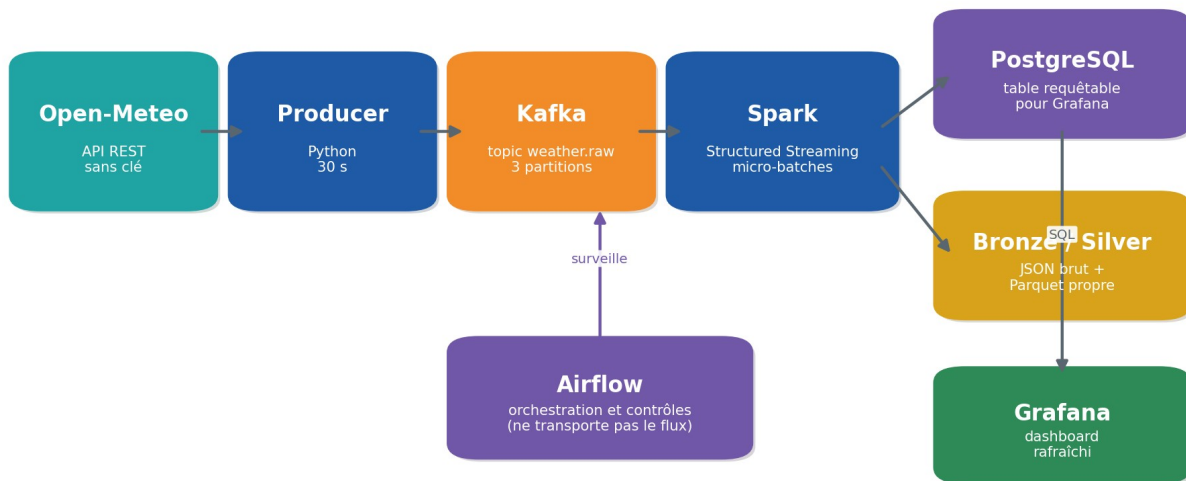
Vue synthétique de la chaîne réalisée.

CHAPITRE 1

Vue d'ensemble du projet

Le projet est une plateforme de collecte et de visualisation météorologique en quasi temps réel. Il utilise l'API Open-Meteo comme source gratuite, sans clé d'API dans la configuration actuelle.

Architecture du projet



Architecture complète du pipeline.

La chaîne en une phrase

Résumé

Un programme Python récupère la météo, Kafka transporte les événements, Spark les nettoie et les stocke, PostgreSQL les rend interrogeables, Grafana les visualise, et Airflow vérifie régulièrement que les différentes étapes sont opérationnelles.

Pourquoi ce projet est intéressant

- Il relie plusieurs technologies au lieu de présenter un outil isolé.
- Il produit un résultat observable : les données sont visibles dans Kafka UI, DBeaver et Grafana.
- Il met en pratique une architecture de streaming, une base SQL, une orchestration et la conteneurisation.
- Il est reproductible sur une autre machine grâce à Docker Compose.

CHAPITRE 2

Notions fondamentales du Data Engineering

Pipeline de données

Un pipeline est une succession d'étapes qui déplacent et transforment des données. Il automatise le trajet entre une source et un usage final. Dans ce projet, l'usage final est un dashboard Grafana, mais il pourrait aussi s'agir d'une alerte, d'un modèle de machine learning ou d'un rapport.

ETL et ELT

Approche	Ordre	Idée
ETL	Extract -> Transform -> Load	Les données sont transformées avant leur chargement final.
ELT	Extract -> Load -> Transform	Les données brutes sont chargées, puis transformées dans la plateforme cible.
Ton projet	Hybride	Extraction vers Kafka, transformation Spark, chargement Bronze/Silver/PostgreSQL.

Tu peux présenter ce projet comme un pipeline ETL de streaming. Il conserve aussi une copie brute Bronze, ce qui reprend une logique moderne proche de l'ELT et du data lake.

Batch vs streaming

Batch	Streaming
Traite un ensemble fini : un fichier du jour, un lot de commandes.	Traite un flux potentiellement infini d'événements.
Exécution planifiée : chaque nuit, chaque heure.	Exécution continue ou micro-batches fréquents.
Latence généralement plus élevée.	Latence faible et données rapidement disponibles.

Événement, message et observation

Une observation météo devient un événement. Le message Kafka contient les données de cet événement au format JSON. L'événement possède notamment un `event_id` unique, un `observed_at` fourni par la source et un `ingested_at` correspondant au moment où le producteur l'a collecté.

Vocabulaire essentiel

Terme	Définition simple
Producer	Application qui publie des messages dans Kafka.
Consumer	Application qui lit des messages Kafka.
Topic	Canal logique qui regroupe une famille de messages.
Partition	Sous-division d'un topic permettant de distribuer et ordonner les messages.
Offset	Numéro séquentiel d'un message dans une partition.
Micro-batch	Petit lot de messages traité à intervalle court par Spark.
Checkpoint	État persistant permettant de reprendre un flux sans repartir de zéro.
JDBC	Standard Java utilisé par Spark pour écrire dans PostgreSQL.
DAG	Graphe de tâches Airflow avec dépendances et ordre d'exécution.

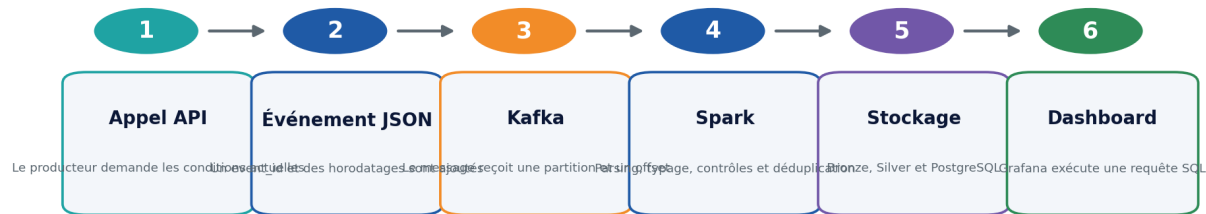
Idempotence

Une opération est idempotente lorsqu'on peut la rejouer sans créer un résultat différent ou des doublons. Dans un pipeline réel, les redémarrages et les réessais sont normaux. La clé primaire `event_id` contribue à empêcher la duplication d'une même observation, mais une stratégie complète peut nécessiter un `UPSERT` ou `ON CONFLICT DO NOTHING`.

CHAPITRE 3

Architecture et cycle de vie d'une donnée

Cycle de vie d'une observation



Résultat : une donnée collectée devient une information visible et exploitable.

Une observation traverse six étapes principales.

Étape 1 - Extraction

Le producteur Python appelle Open-Meteo avec des coordonnées géographiques. Il demande la température, l'humidité, la température ressentie, les précipitations, le code météo et la vitesse du vent.

Étape 2 - Enrichissement technique

Le producteur ajoute un identifiant unique et un horodatage d'ingestion. Ces champs sont essentiels pour tracer la donnée et gérer les doublons.

Étape 3 - Publication Kafka

Le message est publié dans le topic weather.raw. Kafka lui attribue une partition et un offset. Le producteur reçoit un accusé de réception et affiche l'offset dans les logs.

Étape 4 - Transformation Spark

- Lecture du message en chaîne JSON.
- Application d'un schéma explicite.
- Conversion des dates en timestamp.
- Contrôle de température entre -90 et 60 degrés.
- Contrôle d'humidité entre 0 et 100 pour cent.
- Suppression des doublons dans le lot traité.

Étape 5 - Stockage multiple

Spark écrit simultanément dans la zone Bronze, la zone Silver et PostgreSQL. Cette duplication n'est pas inutile : chaque stockage répond à un besoin différent.

Étape 6 - Visualisation

Grafana interroge PostgreSQL avec SQL. Le filtre temporel Grafana limite les lignes à la période visible et le dashboard se rafraîchit automatiquement.

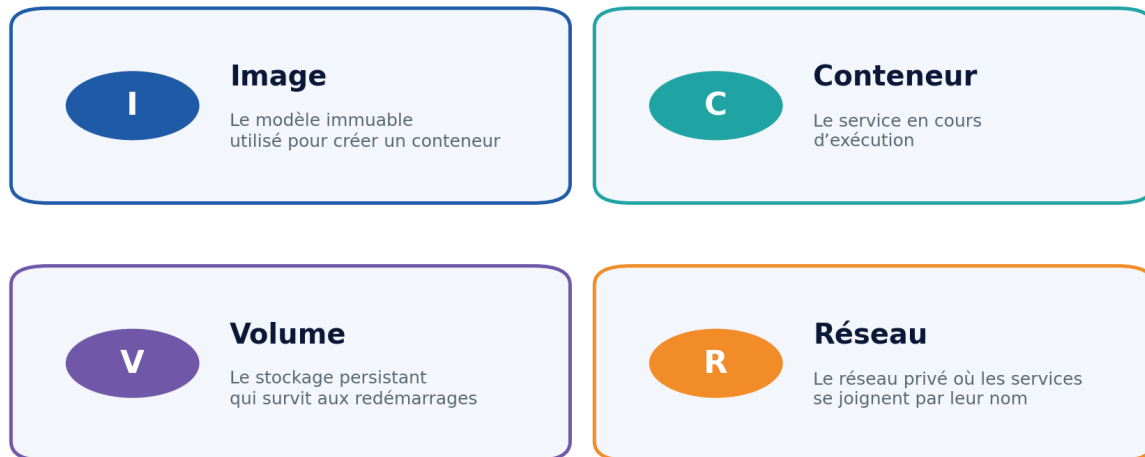
Séparation des responsabilités

Kafka transporte, Spark transforme, PostgreSQL sert les requêtes, Grafana visualise et Airflow orchestre. Aucun outil ne doit tout faire.

CHAPITRE 4

Docker et Docker Compose

Docker : les quatre notions à retenir



Exemple de port : 3001:3000 = port Windows 3001 vers port Grafana 3000 dans le conteneur.

Les concepts Docker à maîtriser.

Image et conteneur

Une image est un modèle versionné. Un conteneur est une instance en cours d'exécution de cette image. Par exemple, postgres:16-alpine est une image et weather-postgres est le conteneur créé à partir d'elle.

Volume

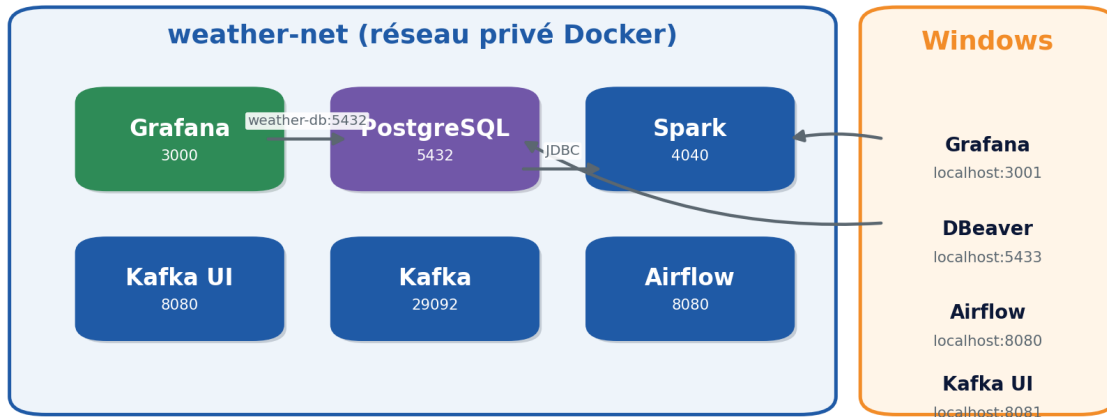
Un conteneur peut être supprimé. Les données importantes doivent donc être enregistrées dans un volume. weather-postgres-data conserve les tables PostgreSQL, grafana-data conserve les dashboards et postgres-data conserve la base interne d'Airflow.

Commande dangereuse

`docker compose down -v` supprime aussi les volumes du projet. La table météo, les métadonnées Airflow et les dashboards Grafana peuvent être perdus. Utilise cette commande uniquement pour une remise à zéro volontaire.

Réseau Docker

Réseau Docker et accès depuis Windows



Différence entre les adresses internes Docker et les ports Windows.

Les services du même réseau se contactent par leur nom de service. Grafana utilise weather-db:5432. DBeaver, installé sur Windows, utilise localhost:5433.

Comprendre le mapping de port

Exemple docker-compose.yml

```
"3001:3000"
```

Le nombre de gauche est le port de la machine Windows. Le nombre de droite est le port dans le conteneur. Tu ouvres donc Grafana sur localhost:3001, même si Grafana écoute sur 3000 à l'intérieur de Docker.

Indentation YAML

YAML utilise l'indentation pour définir la structure. Une erreur d'alignement peut placer weather-db à l'intérieur du service kafka et provoquer une erreur "additional properties not allowed". Les noms de services doivent être alignés sous services:.

Structure correcte

```
services:
  kafka:
    image: ...

  weather-db:
    image: ...
```

Valider le fichier Compose

```
docker compose config
```

Cette commande analyse le YAML et affiche la configuration finale. Utilise-la avant un redémarrage lorsque tu modifies docker-compose.yml.

CHAPITRE 5

Kafka : transport des événements

Pourquoi Kafka ?

Sans Kafka, le producteur pourrait écrire directement dans PostgreSQL. Kafka ajoute toutefois un tampon durable, découple les applications et permet à plusieurs consommateurs de lire le même flux sans modifier le producteur.

Sans Kafka	Avec Kafka
Le producteur dépend directement de la base.	Le producteur publie dans un canal indépendant.
Une panne de la destination peut bloquer l'ingestion.	Les messages restent disponibles pour être consommés plus tard.
Ajouter un nouveau consommateur nécessite souvent de modifier le flux.	Spark, une alerte et un autre service peuvent lire le même topic.

Le topic weather.raw

Le projet crée un topic weather.raw avec trois partitions et un facteur de réplication de 1, adapté à un environnement local avec un seul broker.

Partition et clé

Le producteur utilise la ville comme clé Kafka. Une même clé tend à être envoyée vers la même partition, ce qui aide à préserver l'ordre des événements d'une ville.

Offset

L'offset augmente pour chaque message de la partition. Lorsque les logs affichent offset=0, puis 1, 2 et 3, cela prouve que Kafka accepte les événements.

Kafka UI

- Ouvre <http://localhost:8081>.
- Sélectionne le cluster weather-local.
- Ouvre le topic weather.raw.
- Consulte Messages pour voir le JSON et vérifier que le compteur augmente.

Lire les messages en ligne de commande

```
docker compose exec kafka kafka-console-consumer \  
  --bootstrap-server kafka:29092 \  
  --topic weather.raw \  
  --from-beginning \  
  --max-messages 5
```

À retenir

Kafka ne transforme pas les données. Il les reçoit, les ordonne dans des partitions et les met à disposition des consommateurs.

CHAPITRE 6

Spark Structured Streaming et architecture Bronze/Silver

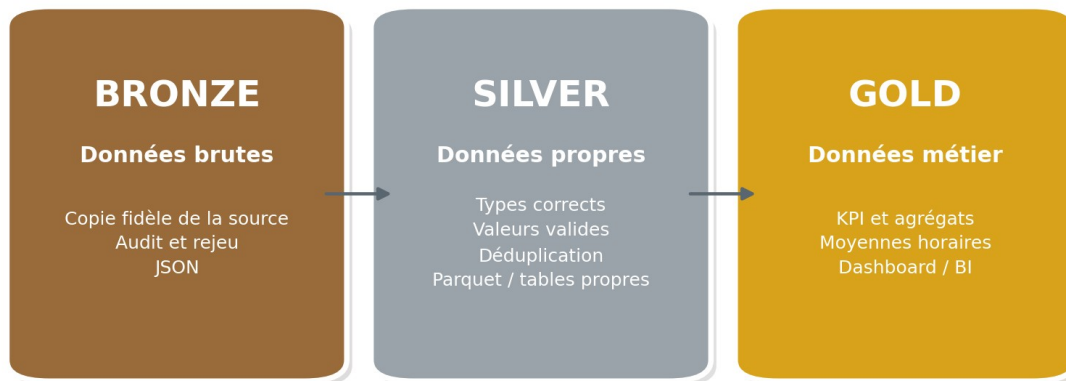
Structured Streaming

Spark Structured Streaming permet de décrire un traitement de flux avec les mêmes concepts que les DataFrames. Le moteur exécute le plan sous forme de micro-batches et suit l'avancement grâce aux checkpoints.

Pourquoi deux écritures ?

Le projet démarre deux requêtes de streaming : une pour Bronze et une pour Silver/PostgreSQL. Elles partagent la lecture logique de Kafka mais disposent de checkpoints différents.

Architecture Medallion



Bronze, Silver et Gold répondent à des usages différents.

Bronze

- Conserve la clé Kafka, le JSON brut, la partition, l'offset et le timestamp Kafka.
- Permet un audit ou un retraitement ultérieur.
- Écrit des fichiers JSON partitionnés par partition Kafka.

Silver

- Parse le JSON avec un schéma connu.
- Convertit les dates et ajoute processed_at.
- Filtre les valeurs manifestement invalides.
- Écrit du Parquet partitionné par ville et date.

PostgreSQL

La même version nettoyée est écrite dans `weather_observations`. PostgreSQL n'est pas la couche Bronze : il s'agit d'une couche de service destinée à SQL et Grafana.

Checkpoint

Les checkpoints enregistrent les offsets déjà traités et l'état du streaming. Si Spark redémarre avec les mêmes checkpoints, il reprend normalement à partir de sa dernière progression.

startingOffsets=earliest

Cette option s'applique principalement lorsque la requête ne possède pas encore de checkpoint. Dès qu'un checkpoint existe, Spark reprend selon cet état et non selon l'option de départ.

Warnings KafkaDataConsumer

Le message "KafkaDataConsumer is not running in UninterruptibleThread" est souvent un avertissement bruyant et non bloquant. La validation réelle consiste à vérifier que le conteneur reste Up et que des lignes apparaissent dans PostgreSQL.

CHAPITRE 7

Lancer le projet

Prérequis

- Docker Desktop lancé.
- Le terminal placé dans le dossier contenant docker-compose.yml.
- Les ports nécessaires disponibles ou adaptés dans le fichier Compose.
- Un fichier .env présent si tu utilises des paramètres personnalisés.

Démarrage standard

Commande principale

```
docker compose up -d
```

L'option -d signifie detached : les conteneurs tournent en arrière-plan et le terminal redevient disponible.

Premier démarrage ou modification des images

```
docker compose up --build -d
```

L'option --build reconstruit les images personnalisées du producer, de Spark et d'Airflow avant le démarrage.

Reconstruction ciblée de Spark

```
docker compose stop spark-streaming
docker compose build --no-cache spark-streaming
docker compose up -d spark-streaming
```

Cette séquence est utile après une modification du Dockerfile Spark ou de weather_stream.py. --no-cache force Docker à ne pas réutiliser une ancienne couche.

Contrôle immédiat

```
docker compose ps
```

État	Interprétation
Up	Le service fonctionne.

État	Interprétation
healthy	Le healthcheck est réussi.
Exited	Le processus principal s'est arrêté.
Restarting	Le service plante et Docker le relance en boucle.
unhealthy	Le conteneur tourne mais le contrôle de santé échoue.

Ordre logique de démarrage

1. Kafka et PostgreSQL démarrent.
2. kafka-init crée le topic.
3. Le producer et Spark se lancent.
4. Airflow initialise sa base et démarre le webserver et le scheduler.
5. Grafana attend PostgreSQL puis démarre.

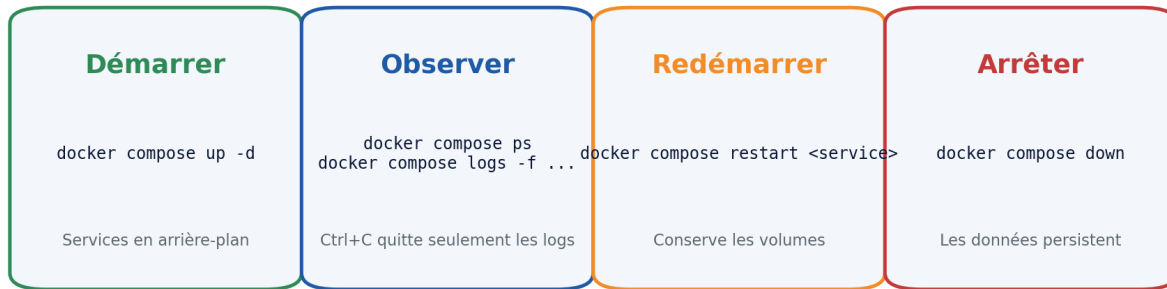
Routine recommandée

Après chaque démarrage : docker compose ps, puis les logs du producer et de Spark, puis une requête SQL dans DBeaver.

CHAPITRE 8

Exploiter, arrêter et redémarrer

Exploitation quotidienne



Les quatre gestes principaux au quotidien.

Suivre les logs

```
docker compose logs -f weather-producer
docker compose logs -f spark-streaming
```

-f signifie follow. Le terminal affiche les nouvelles lignes en continu.

Ctrl+C

Lorsque tu suis les logs avec `docker compose logs -f`, Ctrl+C arrête seulement l'affichage. Le conteneur continue de fonctionner en arrière-plan.

Voir les dernières lignes uniquement

```
docker compose logs --tail=200 spark-streaming
```

Arrêter un service

```
docker compose stop spark-streaming
```

Redémarrer un service

```
docker compose restart weather-producer
```

Recréer un service

```
docker compose up -d --force-recreate grafana
```

Arrêter toute la stack

```
docker compose down
```

Cette commande supprime les conteneurs et le réseau créé par Compose, mais conserve les volumes nommés. Les données PostgreSQL et les dashboards Grafana restent disponibles au prochain lancement.

Remise à zéro totale

À utiliser avec prudence

```
docker compose down -v
```

Cette commande supprime les volumes. Utilise-la seulement si tu veux repartir d'une base vide.

Fermer Docker Desktop

Si Docker Desktop est fermé, tous les conteneurs s'arrêtent. Les volumes restent sur le disque. Au prochain lancement de Docker Desktop, exécute à nouveau `docker compose up -d`.

CHAPITRE 9

PostgreSQL, DBeaver et SQL utile

Deux bases PostgreSQL distinctes

Service	Rôle	Accès
postgres	Métadonnées internes Airflow	Ne pas utiliser pour les données météo.
weather-db	Données métier météo	DBeaver, Spark et Grafana.

Connexion DBeaver

Paramètre	Valeur
Host	localhost
Port	5433
Database	weather
Username	weather_user
Password	weather_password

La table weather_observations

Colonne	Rôle
event_id	Identifiant unique, clé primaire.
city	Ville associée aux coordonnées.
observed_at	Heure de l'observation selon la source.
ingested_at	Heure de collecte par le producer.
processed_at	Heure de traitement par Spark.
temperature_2m	Température à deux mètres.
relative_humidity_2m	Humidité relative.
apparent_temperature	Température ressentie.

Colonne	Rôle
precipitation	Précipitation.
weather_code	Code météo.
wind_speed_10m	Vitesse du vent.

Requêtes indispensables

Voir les dernières observations

```
SELECT *
FROM weather_observations
ORDER BY ingested_at DESC;
```

Compter les lignes

```
SELECT COUNT(*)
FROM weather_observations;
```

Statistiques par ville

```
SELECT city, MAX(temperature_2m), MIN(temperature_2m), AVG(temperature_2m)
FROM weather_observations
GROUP BY city;
```

Dernière heure

```
SELECT *
FROM weather_observations
WHERE ingested_at >= NOW() - INTERVAL '1 hour'
ORDER BY ingested_at;
```

Index conseillé

```
CREATE INDEX IF NOT EXISTS idx_weather_observed_at
ON weather_observations(observed_at);
```

Cet index accélère les requêtes temporelles de Grafana lorsque la table devient volumineuse.

Contraintes et doublons

event_id est une clé primaire. Une tentative d'insertion du même identifiant échoue. Pour un pipeline robuste, une écriture avec ON CONFLICT DO NOTHING ou un mécanisme d'upsert évite qu'un doublon fasse échouer tout un micro-batch.

CHAPITRE 10

Grafana et dashboards temps réel

Connexion PostgreSQL dans Grafana

Champ	Valeur
Host URL	weather-db:5432
Database	weather
User	weather_user
Password	weather_password
TLS/SSL mode	disable
PostgreSQL version	16

Pourquoi pas localhost:5433 ?

Grafana est dans Docker. Il utilise le réseau interne et contacte le service weather-db sur son port interne 5432. localhost désignerait le conteneur Grafana lui-même.

Premier panneau Time series

```
SELECT
    observed_at AS "time",
    temperature_2m AS temperature
FROM weather_observations
WHERE $__timeFilter(observed_at)
ORDER BY observed_at;
```

Le macro `$__timeFilter` est remplacé automatiquement par Grafana selon la période sélectionnée, par exemple les six dernières heures.

Panneau Stat - température actuelle

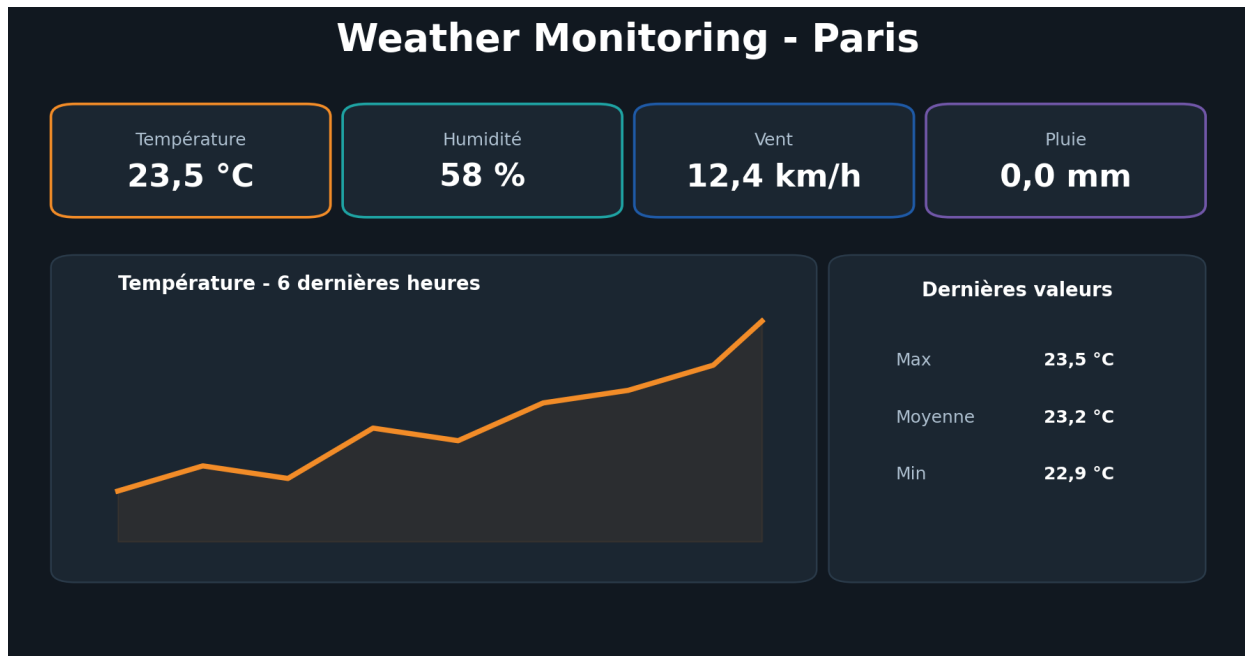
```
SELECT
    observed_at AS "time",
    temperature_2m AS value
FROM weather_observations
WHERE $__timeFilter(observed_at)
ORDER BY observed_at DESC
LIMIT 1;
```


Humidité, vent et précipitations

Duplique le panneau Stat et remplace temperature_2m par relative_humidity_2m, wind_speed_10m ou precipitation. Configure ensuite l'unité dans Standard options : celsius, percent, km/h ou millimeters.

Rafraîchissement automatique

- Sélectionne une période courte, par exemple Last 6 hours.
- Choisis un intervalle de refresh de 30 secondes.
- Évite un refresh beaucoup plus rapide que la fréquence d'ingestion.



Exemple de composition d'un dashboard métier.

Design recommandé

- Première ligne : température, humidité, vent, pluie.
- Deuxième ligne : courbe de température et de température ressentie.
- Troisième ligne : précipitations et vitesse du vent.
- Titre clair, unités visibles et période temporelle cohérente.

Persistance des dashboards

Les dashboards sont enregistrés dans le volume grafana-data. docker compose down les conserve.
docker compose down -v les supprime.

CHAPITRE 11

Airflow et orchestration

Streaming continu vs orchestration Airflow



Le streaming et Airflow ont des rôles complémentaires.

Le DAG weather_pipeline_monitoring

Le DAG s'exécute toutes les dix minutes et enchaîne quatre contrôles.

1. check_open_meteo_api : vérifie que la source répond.
2. ensure_kafka_topic : vérifie ou crée le topic Kafka.
3. check_recent_kafka_event : attend un nouvel événement.
4. check_silver_output : vérifie qu'un fichier Parquet récent existe.

Activer et déclencher

- Ouvre `http://localhost:8080`.
- Active le commutateur du DAG.
- Clique sur le bouton de déclenchement manuel pour tester.
- Ouvre la vue Grid ou Graph pour suivre les tâches.

Broken DAG

Un DAG cassé n'apparaît pas dans la liste normale. Airflow affiche une erreur d'import. La commande suivante montre la cause exacte.

```
docker compose exec airflow-scheduler airflow dags list-import-errors
```

Ce qu’Airflow ne fait pas

Airflow n’est pas le moteur qui traite chaque événement météo. Kafka et Spark assurent le flux continu. Airflow exécute des vérifications planifiées, des opérations de maintenance ou des traitements batch complémentaires.

CHAPITRE 12

Vérifier le pipeline de bout en bout

Checklist en cinq minutes

1. docker compose ps : les services critiques sont Up.
2. Logs producer : les offsets augmentent.
3. Kafka UI : weather.raw contient des messages.
4. DBeaver : weather_observations contient de nouvelles lignes.
5. Grafana : la courbe affiche ces mêmes données.

Validation du producer

```
docker compose logs --tail=30 weather-producer
```

Cherche “Connexion Kafka établie” puis “Événement envoyé” avec un offset croissant.

Validation de Kafka

Dans Kafka UI, le nombre de messages doit augmenter. Un topic présent mais vide indique que Kafka fonctionne probablement mais que le producer n’envoie rien.

Validation de Spark

```
docker compose ps spark-streaming
docker compose logs --tail=100 spark-streaming
```

Le service doit rester Up. Des warnings peuvent apparaître ; cherche surtout ERROR, Exception et Caused by.

Validation PostgreSQL

```
SELECT COUNT(*), MAX(ingested_at)
FROM weather_observations;
```

Le compteur doit augmenter et MAX(ingested_at) doit être récent.

Validation Grafana

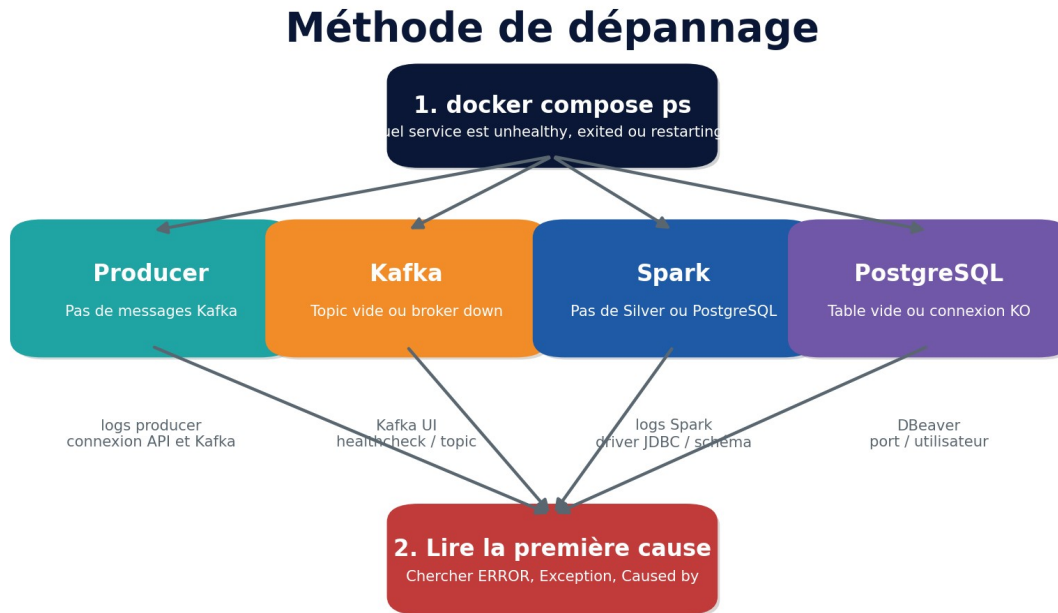
Sélectionne une période contenant les observations et clique sur Refresh. Si DBeaver montre les lignes mais pas Grafana, le problème se situe dans la source de données, le filtre temporel ou la requête du panneau.

Latences à observer

Horodatage	Signification
observed_at	Moment de la mesure fournie par Open-Meteo.
ingested_at	Moment où le producer collecte la réponse.
processed_at	Moment où Spark traite le message.
kafka_timestamp	Moment où Kafka reçoit le message.

CHAPITRE 13

Dépannage complet



Toujours isoler le composant avant de modifier le code.

Méthode générale

1. Identifier le service en défaut avec docker compose ps.
2. Lire les logs de ce seul service.
3. Chercher la première exception, pas uniquement les dernières lignes de la stack trace.
4. Vérifier les dépendances : réseau, port, nom de service, base, driver.
5. Corriger, reconstruire uniquement le service concerné et vérifier à nouveau.

Erreur : image Bitnami Spark introuvable

Symptôme : bitnami/spark:3.5.3 not found. Correction appliquée : utiliser apache/spark:3.5.3 et le chemin /opt/spark/bin/spark-submit.

Erreur : kafka.vendor.six.moves

Symptôme : ModuleNotFoundError avec kafka-python sous Python 3.12. Correction appliquée dans Airflow et le producer : remplacer kafka-python par kafka-python-ng tout en conservant les imports from kafka.

Erreur : Broken DAG

Utilise la bannière Airflow ou `airflow dags list-import-errors`. Reconstructs les images Airflow après avoir modifié `requirements.txt`.

Erreur YAML : additional properties weather-db not allowed

weather-db avait été indenté à l'intérieur de kafka. Aligne tous les services sous `services:` et valide avec `docker compose config`.

Erreur : port 3000 déjà utilisé

Le port Windows était occupé. Le mapping Grafana a été modifié en `3001:3000`. L'URL est donc `localhost:3001`.

Erreur : driver PostgreSQL absent

Les logs Spark ne montraient que le package Kafka. Le package `org.postgresql:postgresql:42.7.5` a été ajouté à `--packages`, puis l'image Spark reconstruite sans cache.

Table PostgreSQL vide

- Vérifier que Spark est Up et non Restarting.
- Vérifier que les variables `POSTGRES_URL`, `USER`, `PASSWORD` et `TABLE` sont définies.
- Vérifier que weather-db est healthy.
- Chercher une `SQLException` ou une erreur de type.
- Vérifier que le checkpoint ne bloque pas un rejeu attendu.

Grafana : Database Connection OK mais pas de courbe

- Sélectionner une période qui contient les timestamps.
- Utiliser `observed_at AS "time"`.
- Utiliser `$_timeFilter(observed_at)`.
- Cliquer sur Run query et sélectionner Time series.

Commandes de diagnostic

```
docker compose ps
docker compose logs --tail=200 <service>
docker compose logs --tail=300 spark-streaming | grep -Ei "error|exception|caused by|
psqlexception"
docker compose config
docker stats
```

CHAPITRE 14

Qualité, doublons, sécurité et bonnes pratiques

Qualité des données

Les contrôles actuels filtrent les températures et l'humidité hors limites. Un pipeline plus avancé vérifierait aussi la présence des champs obligatoires, les valeurs de vent, les timestamps dans le futur et le taux de données manquantes.

Déduplication

dropDuplicates dans foreachBatch agit sur le lot courant. Il ne garantit pas à lui seul l'absence de doublons entre plusieurs exécutions. La clé primaire event_id aide, mais une écriture JDBC standard peut faire échouer le batch lors d'un conflit.

Approche robuste

- Écrire dans une table de staging.
- Exécuter INSERT ... ON CONFLICT DO NOTHING vers la table finale.
- Ou utiliser une fonction PostgreSQL appelée à chaque micro-batch.
- Surveiller le nombre de doublons rejetés.

Secrets et GitHub

Les mots de passe du projet sont volontairement simples pour un usage local. Pour publier le dépôt, place les secrets dans .env, conserve seulement .env.example et ajoute .env au .gitignore.

Ne jamais publier

Une vraie clé API, un mot de passe de production, un token cloud, un certificat privé ou une URL contenant des identifiants.

Versions Docker

Évite latest dans un projet reproductible. Épinglez une version de Grafana après validation. Une image latest peut changer et casser le comportement lors d'un futur pull.

Principe du moindre privilège

Grafana devrait idéalement utiliser un utilisateur PostgreSQL en lecture seule. Spark utiliserait un utilisateur autorisé à insérer, tandis qu'un compte d'administration resterait réservé à la maintenance.

Observabilité

- Logs structurés et lisibles.
- Métriques sur le nombre de messages et la latence.
- Alertes si le dernier événement est ancien.
- Suivi du consumer lag Kafka.
- Contrôle de l'espace disque des volumes.

CHAPITRE 15

Couche Gold et prochaines évolutions

Qu'est-ce que Gold ?

Gold contient des données directement alignées sur une question métier. Il ne s'agit plus d'observations unitaires, mais de moyennes, maximums, tendances ou alertes prêtes pour les utilisateurs.

Exemple : vue horaire

```
CREATE MATERIALIZED VIEW weather_hourly AS
SELECT
  city,
  date_trunc('hour', observed_at) AS hour,
  AVG(temperature_2m) AS avg_temperature,
  MAX(temperature_2m) AS max_temperature,
  MIN(temperature_2m) AS min_temperature,
  AVG(relative_humidity_2m) AS avg_humidity,
  MAX(wind_speed_10m) AS max_wind
FROM weather_observations
GROUP BY city, date_trunc('hour', observed_at);
```

Multi-villes

Le producer actuel suit une ville. Une évolution consiste à parcourir une configuration de villes, publier chaque ville avec sa clé Kafka et ajouter une variable city dans Grafana.

Alertes

- Grafana : alerte si température supérieure à un seuil.
- Spark : publier un événement dans un topic weather.alerts.
- Airflow : vérifier qu'aucune ville n'est silencieuse depuis trop longtemps.

Stockage analytique

Pour un volume plus important, PostgreSQL peut être complété ou remplacé par ClickHouse, TimescaleDB, BigQuery, Snowflake ou un lakehouse. Le choix dépend du volume, de la latence, du budget et des usages.

Tests et CI/CD

- Tests unitaires du parsing et des règles de qualité.
- Test d'intégration avec un message Kafka de référence.
- Validation docker compose config dans GitHub Actions.
- Build automatique des images.
- Lint Python et vérification des dépendances.

Roadmap raisonnable

Niveau	Évolution
1	Dashboard complet avec quatre indicateurs et trois séries.
2	Multi-villes et variable Grafana.
3	Vue Gold horaire et index PostgreSQL.
4	Alertes et contrôle de fraîcheur.
5	Tests, CI GitHub Actions et documentation publique.
6	Déploiement cloud et monitoring technique.

CHAPITRE 16

Présenter le projet sur GitHub et en entretien

Pitch de 30 secondes

Exemple

J'ai construit un pipeline météo near real-time conteneurisé. Un producer Python collecte Open-Meteo, publie les observations dans Kafka, Spark Structured Streaming les valide et les écrit en Bronze, Silver et PostgreSQL. Grafana affiche les données et Airflow supervise la santé du pipeline.

Pitch de deux minutes

Explique le besoin, puis l'architecture, puis les choix. Commence par la valeur : rendre des observations rapidement visibles. Kafka découple l'ingestion du traitement. Spark applique le schéma et les règles de qualité. La zone Bronze permet le rejeu, Silver fournit une donnée propre, PostgreSQL sert les requêtes et Grafana matérialise le résultat. Airflow ne gère pas chaque événement : il orchestre les contrôles planifiés.

Questions fréquentes

Question	Réponse attendue
Pourquoi Kafka ?	Découplage, tampon durable, plusieurs consommateurs, reprise.
Pourquoi Spark ?	Traitement distribué, Structured Streaming, schéma et micro-batches.
Pourquoi PostgreSQL si tu as Parquet ?	Grafana et les utilisateurs SQL ont besoin d'une couche facilement interrogeable.
Pourquoi Bronze et Silver ?	Conserver le brut puis fournir une donnée propre et reproductible.
Est-ce du vrai temps réel ?	Near real-time, avec polling et micro-batches.
Quel rôle joue Airflow ?	Orchestration et supervision, pas transport du flux.
Comment éviter les doublons ?	Clé unique, checkpoint et upsert/ON CONFLICT.
Que se passe-t-il si Spark tombe ?	Kafka conserve les messages et le checkpoint permet la reprise.

Structure d'un README

1. Titre et phrase de valeur.

2. Schéma d'architecture.
3. Technologies utilisées.
4. Prérequis et démarrage rapide.
5. URLs et identifiants de démonstration.
6. Captures Kafka UI, Airflow, DBeaver et Grafana.
7. Explication Bronze/Silver/Gold.
8. Difficultés rencontrées et solutions.
9. Roadmap.

Formulation CV

Conception d'un pipeline de données near real-time avec Kafka, Spark Structured Streaming, Airflow, PostgreSQL et Grafana pour ingérer une API REST, appliquer des contrôles de qualité, stocker les données dans une architecture Bronze/Silver et alimenter un dashboard conteneurisé avec Docker Compose.

Ce qu'il faut réellement comprendre

Un entretien ne demande pas seulement de réciter des commandes. Il faut savoir expliquer le trajet d'un événement, les responsabilités des services, le rôle des volumes, les risques de doublons et la manière dont tu localises une panne.

CHAPITRE 17

Fiches mémo et glossaire

Fiche mémo - démarrage

```
cd ".../RT Pipeline"
docker compose config
docker compose up -d
docker compose ps
```

Fiche mémo - accès

Outil	URL ou connexion
Airflow	http://localhost:8080
Kafka UI	http://localhost:8081
Spark UI	http://localhost:4040
Grafana	http://localhost:3001
DBEaver	localhost:5433 / weather / weather_user

Fiche mémo - logs

```
docker compose logs -f weather-producer
docker compose logs -f spark-streaming
docker compose logs -f airflow-scheduler
docker compose logs -f grafana
```

Fiche mémo - arrêt

```
Ctrl+C # quitte seulement logs -f
docker compose stop <service> # arrête un service
docker compose down # arrête toute la stack
docker compose down -v # supprime aussi les données
```

Fiche mémo - reconstruction

```
docker compose build --no-cache spark-streaming
docker compose up -d spark-streaming
```

Fiche mémo - SQL

```
SELECT COUNT(*), MAX(ingested_at)
FROM weather_observations;
```

```
SELECT *
FROM weather_observations
ORDER BY ingested_at DESC
LIMIT 20;
```

Glossaire

Terme	Définition
API	Interface permettant à une application de demander des données à un service.
Broker	Serveur Kafka qui stocke les partitions.
Consumer lag	Écart entre les messages disponibles et ceux déjà traités par un consommateur.
Container	Processus isolé créé depuis une image Docker.
DAG	Ensemble de tâches Airflow et de leurs dépendances.
Data lake	Stockage de fichiers de données brutes et transformées.
Data warehouse	Base organisée pour l'analyse et le reporting.
Event time	Temps associé à l'événement, ici <code>observed_at</code> .
Ingestion time	Temps auquel le système reçoit la donnée, ici <code>ingested_at</code> .
JDBC	Connecteur standard utilisé par Spark pour les bases SQL.
Near real-time	Données disponibles avec une faible latence, sans garantie milliseconde.
Offset	Position d'un message dans une partition Kafka.
Parquet	Format colonne performant pour l'analyse.
Partition	Découpage logique ou physique des données.
Schema	Description des colonnes et de leurs types.
Streaming	Traitement continu d'un flux d'événements.
Topic	Canal Kafka contenant une catégorie de messages.
Volume	Stockage Docker persistant.

Terme	Définition
Watermark	Mécanisme de gestion du retard d'événements dans certains traitements de streaming.
Upsert	Insertion ou mise à jour selon l'existence de la clé.

Dernière règle

Quand quelque chose ne marche pas : ne modifie pas tout. Identifie le premier service cassé, lis sa première erreur, puis corrige une seule couche à la fois.

Conclusion

Tu es parti d'une API gratuite et tu as construit une chaîne complète, observable et reproductible. La compétence principale n'est pas d'avoir utilisé huit logos : c'est de comprendre comment une donnée traverse les composants, comment chaque composant assume une responsabilité claire, et comment vérifier ou réparer le système sans tout recommencer.

Le projet est déjà présentable. Les prochaines améliorations - Gold, multi-villes, alertes, tests et CI/CD - servent à approfondir les mêmes principes : qualité, reprise, observabilité, sécurité et valeur métier.

Projet terminé, apprentissage ouvert

Garde ce manuel avec le dépôt. Le jour où tu relances le projet, commence par la fiche mémo, puis utilise le chapitre de dépannage si un service ne démarre pas.